convenient-karnaughs

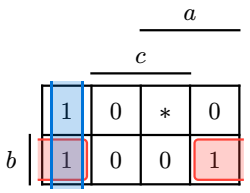
This is the manual containing the full API documentation. But first a few examples to illustrate some common use cases.

A karnaugh map for a truth table:

a	b	c	f
0	0	0	**1**
0	0	1	**0**
0	1	0	**1**
0	1	1	**0**
1	0	0	**0**
1	0	1	$*$
1	1	0	**1**
1	1	1	**0**

```
#import "@preview/convenient-karnaughs:1.0.0": karnaugh-map

#karnaugh-map(
  ($a$, $b$, $c$),
  (1, 0, 1, 0, 0, -1, 1, 0),
  // ↑ truth table order
  implicants: (
    (a, b, c) => not a and not c,
    // ↑ Typst functions
    (a, b, c) => b and not c,
    // ↑ borders don't overlap
  ),
)
```

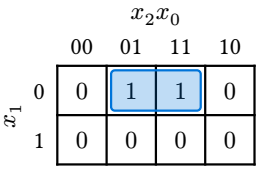


A karnaugh map for a function and with different label styles:

$$f(x_2, x_1, x_0) = \neg x_1 \wedge x_0$$

```
#import "@preview/convenient-karnaughs:1.0.0": karnaugh-map

#karnaugh-map(
  3,
  (x2, x1, x0) => not x1 and x0,
  // ↑ Typst functions
  implicants: (
    (-1, 0, 1),
    // ↑ variable assignments
  ),
  style: (labels: "bitstring"),
)
```



Functions

- karnaugh-map()

Variables

- default-style

Types

- bool-like

Function `karnaugh-map`

A Karnaugh map.

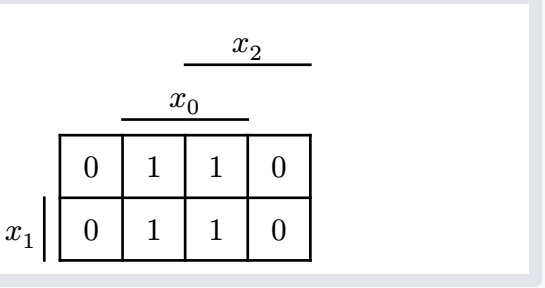
Parameters

```
karnaugh-map(
  vars: int array,
  f: function array,
  implicants: array,
  style: dictionary
)
```

vars `int` or `array`

The number or names of the variables, of which there can be arbitrarily many.

```
karnaugh-map(
  3,
  // or
  ($x_2$, $x_1$, $x_0$),
  (x2, x1, x0) => x0,
)
```

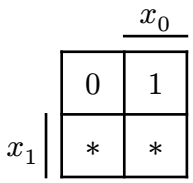


f `function` or `array`

The function or an array of the boolean-like function values. If an array, the order is read off of a truth table where the last variable changes the fastest.

x_1	x_0	f
0	0	**0**
0	1	**1**
1	0	*
1	1	*

```
karnaugh-map(
  2,
  (x1, x0) =>
    if x1 {none} else {x0},
  // or
  (false, true, none, none),
  // or
  (0, 1, -1, -1),
)
```

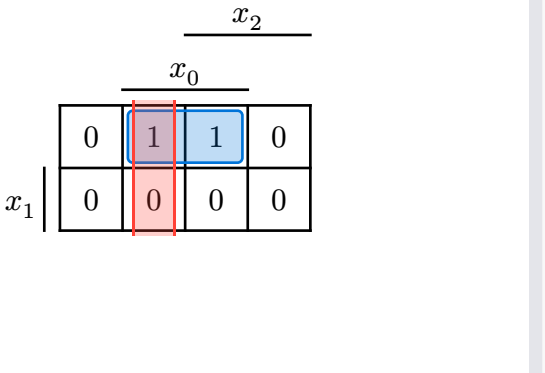


implicants `array`

A list of implicants, which can either be functions or arrays of boolean-like values representing variable assignments.

They are automatically rendered so that no two borders are directly on top of each other. It even finds the layout with the minimal amount of adjustments.

```
karnaugh-map(
  3,
  (0, 1, 0, 0, 0, 1, 0, 0),
  implicants: (
    (x2, x1, x0) => not x1 and x0,
    // or
    (none, false, true),
    // or
    (-1, 0, 1),
    (x2, x1, x0) => not x2 and x0,
    // ↑ automatically adjusted
  )
)
```



Default: `()`

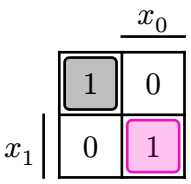
style `dictionary`

Styling configuration that is merged with the defaults. The dictionary may contain the following keys:

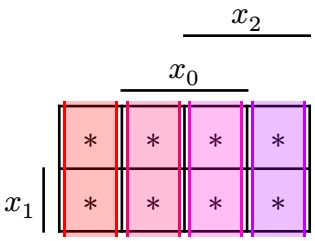
style.implicant-colors `array` or `function`

Either a list of colors or a function that calculates a color from the implicant index. There must be at least as many colors as implicants.

```
karnaugh-map(
  2,
  (1, 0, 0, 1),
  implicants: (
    (x1, x0) => not x1 and not x0,
    (x1, x0) => x1 and x0,
  ),
  style: (
    implicant-colors: (black, fuchsia),
  )
)
```



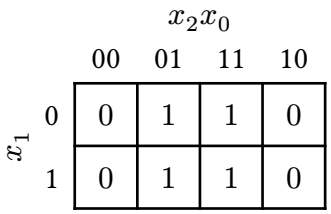

```
karnaugh-map(
  3,
  (x2, x1, x0) => none,
  implicants: (
    (x2, x1, x0) => not x0 and not x2,
    (x2, x1, x0) => x0 and not x2,
    (x2, x1, x0) => x0 and x2,
    (x2, x1, x0) => not x0 and x2,
  ),
  style: (
    implicant-colors: (i) => {
      color.hsl(-25deg * i, 100%, 50%)
    },
  ),
)
```



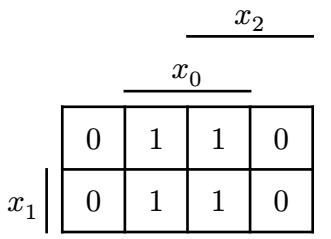
style.labels `"bitstring"` or `"line"` or `function`

Values `"bitstring"` and `"line"` are presets, but a custom function can be provided. It takes the variables and the function grid as arguments, where each function grid cell is `1em` wide.

```
karnaugh-map(
  3,
  (x2, x1, x0) => x0,
  style: (
    labels: "bitstring",
  ),
)
```




```
karnaugh-map(
  3,
  (x2, x1, x0) => x0,
  style: (
    labels: "line",
  ),
)
```



Default: `(`
`  implicant-colors: (blue, red, green, purple, yellow, teal, black),`
`  labels: "line",`
`)`

Variable `default-style` `dictionary`

Default value of `karnaugh-map.style` exposed as variable. Use this only in case you want to change it and export it as variable. Otherwise just pass a partial styles object into `karnaugh-map()`.

```
let my-style = (
  ..default-style,
  // Overrides here
)
```

Type `bool-like`

A type representing a boolean or “don’t care”. Anywhere this type is required, one of the values `true`, `false`, `none`, `1`, `0`, or `-1` is expected.

The values `true` and `false` represent the respective boolean values while `none` represents a “don’t care”. For convenience, the integers `1`, `0`, and `-1` can be used, respectively.